



ВТУ «Св. Св. Кирил и Методий»

Факултет «Математика и информатика»

РЕФЕРАТ

*Структурни шаблони за проектиране
(Structural design patterns)*

Разработил:

Николета Панайотова

Спец. “Информационни ситеми”

Фак. №: МПИ-10679р

Проверил:

гл. ас. д-р Ивайло Дончев

Велико Търново
2012г.

Съдържание

<u>Увод</u>	3
Структурни шаблони (<i>Structural Design Patterns</i>).....	4
1. Адаптер (<i>Adapter</i>)	4
2. Мост (<i>Bridge</i>)	6
3. Композиция (<i>Composite</i>)	9
4. Декоратор (<i>Decorator</i>)	11
5. фасада (<i>Facade</i>)	13
6. Миниобект (<i>Flyweight</i>)	15
7. Пълномощно (<i>Proxy</i>)	17
<u>Заклучение</u>	20
Използвана литература	20

Увод

Създаването на дизайн и проектирането чрез обектно-ориантирани софтуери е много трудно, обектите трябва да се преобразуват в класове, да се дефинират интерфейсни класове с наследяването и да се определи клоч, който да ги обединява. Дизайна трябва да бъде специфичен и конкретен на изискването но и също така да позволи бъдещи разширения и обновявания за да се избягва изцяло проектиране *ри* нужда.

Шаблоните за проектиране могат да бъдат разделени на три вида като например създаващи (*Creational*), структурни (*Structural*) и поведенчески (*Behavioral*).

В настоящият реферат ще се разгледат единствено структурните шаблони за проектиране. Те определят начина, по който класовете и обектите са композирани за да оформят по-големи структури. Структурният клас използва наследяването за да създаде интерфейси или имплементации. Като например да вземем няколко наследявания, които обединяват две или повече класове в един. Резултатът е един клас, който комбинира свойствата на техните класове бащи. Този шаблон е много нужен за да накара независимите разработени класове да работят заедно. Те могат да бъдат реализирани на всички обектно-ориантирани езици като например *Java*, *PHP*, *C++*, *C#* и други.

Структурни шаблони:

Вместо да се създават интерфейси или имплементации, структурния обект шаблон описва начините, по които са съставени обектите за реализирането на нова функционалност. Добавената гъвкавост при съставяне на обектите е направена с цел да има възможност да се промени устройването по време на изпълнение, което е невъзможно при статическите класове.

Структурните шаблони могат да бъдат разделени на:

- **Адаптер (Adapter):** Конвертира интерфейса на даден клас към друг интерфейс, който е очакван от клиента. Адаптерът оставя класовете да работят заедно. Това е необходимо заради несъвместимостта им.
- **Мост (Bridge):** Отделя абстракцията от нейната имплементация, така че двете могат да бъдат променяни независимо.
- **Композиция (Composite):** Съставя обектите в дървовидни структури за да представляват част от цяла йерархия. Композицията позволява на клиента да работи с индивидуалните обекти и композиции от обекти по един и същ начин.
- **Декоратор (Decorator):** Динамично добавя допълнителни отговорности на обект, като запазва интерфейса му. Декораторите предоставят гъвкава алтернатива на наследяването за разширяване на функционалността.
- **Фасада (Facade):** Предоставя уеднаквен интерфейс за редица интерфейси. Фасадата дефинира интерфейс от по-високо ниво, което прави по-лесна употребата на подсистемата.
- **Миниобект (Flyweight):** Използва споделянето за ефективна поддръжката на голям брой обекти.
- **Пълномощно (Proxy):** Предоставя заместник на друг обект, за да се контролира достъпа до него.

1. Адаптер (Adapter)

Конвертира интерфейса на даден клас към друг интерфейс, който е очакван от клиента. Адаптерът оставя класовете да работят заедно. На пример един редактор за рисуване, който позволява на потребителите да нарисуват и организират графични елементи (линии, многоъгълници, текст, и т.н.), така че да се преобразуват в снимки и диаграми. Абстрактният ключ на редактора е графичния обект, който има редактируема

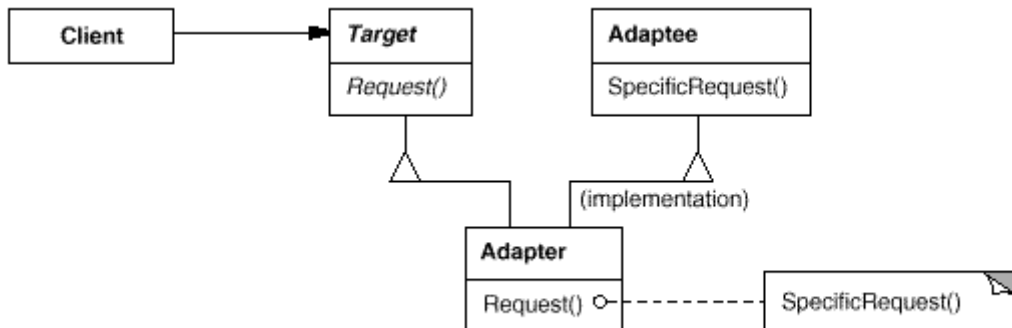
форма. Интерфейса за графичните обекти се определя от абстрактен клас форма (*Shape*). Редактора определя и дефинира един подклас на на класа форма за всеки графичен обект: един линеен-форма клас (*LineShape*) за линии, един полигон-форма клас (*PolygonShape*) за полигони, и така нататък.

Използваемост: Адаптерният шаблон се използва когато:

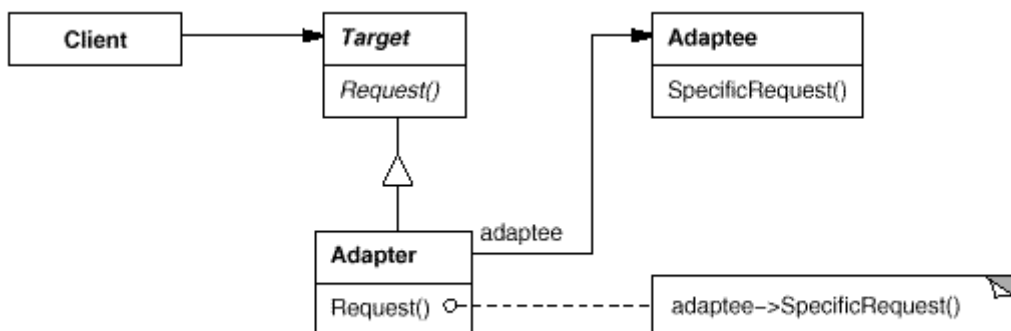
- Искаме да използваме съществуващ клас но неговият интерфейс не съвпада с този, от което се нуждаеме.
- Искаме да създадем клас за многократна употреба, който да сътрудничи с несвързани или непредвидени класове, това са класове, които не е задължително да имат съвместими интерфейси.
- Трябва да използваме няколко съществуващи подкласове, но че е непрактично да се адаптират техните интерфейс като подгрупираме всеки един от тях. Един обект адаптер може да се адаптира интерфейса на клас баща.

Структура:

Един клас адаптер използва множество наследявания за да се адаптира един интерфейс към друг:



Фиг. 1.1 – Диаграма на клас адаптер от структурен шаблон.



Фиг. 1.1 – Диаграма на обект адаптер от структурен шаблон.

Участници:

- **Цел (Shape):** определя интерфейса на специфичния домейн, който ползва клиента.
- **Клиент (DrawingEditor)** сътрудничи с обекти съответстващи с целевият интерфейс.
- **Адапатор (TextView)** определя съществуващ интерфейс, който се нуждае от адаптиране.
- **Адаптер (TextShape)** адаптира интерфейс на Адапатор на целевият интерфейс.

Примерен код:

```
using System;
namespace DoFactory.GangOfFour.Adapter.Structural { // Adapter Design Pattern
    class MainApp {
        static void Main() {
            Target target = new Adapter(); // Create adapter
            target.Request();
            Console.ReadKey(); // Wait for user
        }
    }

    class Target { // The 'Target' class
        public virtual void Request() {
            Console.WriteLine("Called Target Request()");
        }
    }

    class Adapter : Target { // The 'Adapter' class
        private Adaptee _adaptee = new Adaptee();
        public override void Request() {
            _adaptee.SpecificRequest();
        }
    }

    class Adaptee { // The 'Adaptee' class
        public void SpecificRequest() {
            Console.WriteLine("Called SpecificRequest()");
        }
    }
}
```

2. Мост (Bridge)

Отделя абстракцията от нейната имплементация, така че двете могат да бъдат променяни независимо. Да вземем в предвид една имплементация на преносим абстрактен прозорец в набор от инструкции на един потребителски интерфейс. Тази абстракция трябва да ни позволи да пишем приложения, които работят едновременно на X Window System и на IBM Presentation Manager (PM), например.

Използвайки наследяването, бихме могли да определим абстрактен клас *Window* и подкласовете *XWindow* и *PMWindow*, които да реализират *Window* интерфейса за различни платформи.

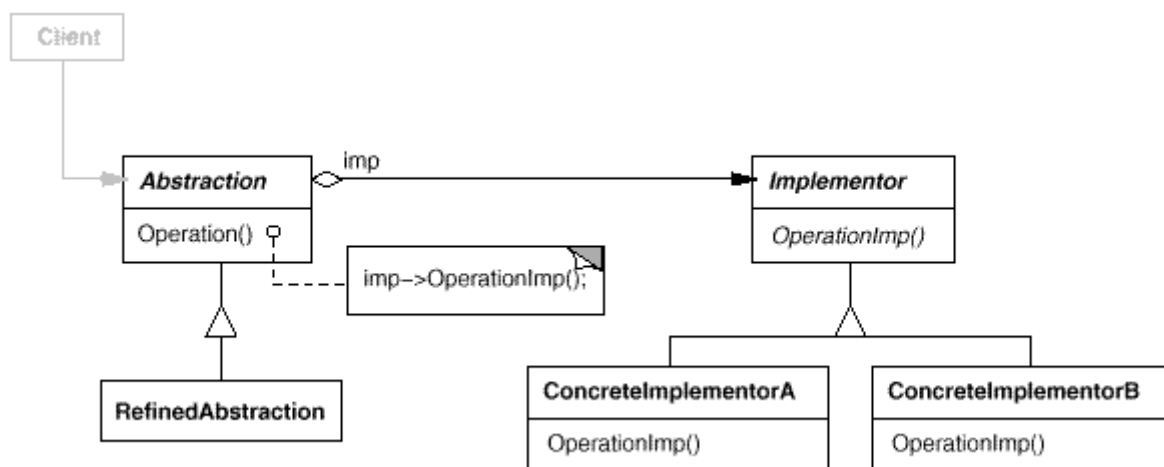
Клиентите трябва да могат създадат прозорец, без да се ангажира с конкретната имплементация. Само изпълнението реализацията на прозореца (*Window*) трябва да зависи от платформата, на която се изпълнява заявката. Затова кодът на клиента трябва да инстанцира прозорци, без да споменава конкретни платформи.

Моделът **мост** се занимава с тези проблеми чрез поставяне на абстракцията прозорец (*Window*) и нейното прилагане в отделни йерархични класове. Има един йерархичен клас за интерфейсите прозорците (*Window*, *IconWindow*, *TransientWindow*) и отделна йерархия за реализация на специфични платформи за прозорци. подкласът *XWindowImp*, например, позволява изпълнение въз основа на X Window System.

Връзката между *Window* и *WindowImp* се определя като мост, защото тя съединява абстракцията и изпълнението и, давайки им възможност да се действат независимо.

Използваемост: Адаптерният шаблон се използва когато:

- Искаме да се избегне трайно обвързване между абстракцията и прилагането и.
- Двете абстракции и тяхното изпълнение трябва да бъде разширяеми от подкласовете.
- Промени в изпълнението на абстракцията не трябва да има никакво влияние върху клиентите, техният код не трябва да бъдат компилирани на ново.



Фиг. 2.1 – Структурата на мост от структурен шаблон.

Участници:

- **Абстракция (Abstraction):** Определя интерфейса на абстракцията и поддържа препратка към обект от тип изпълнител (*Implementor*).
- **Изискана абстракция (RefinedAbstraction):** Разширява определеният интерфейс от абстракцията.
- **Изпълнител (Implementor):** определя интерфейса за изпълнението на класовете. Този интерфейс не трябва да отговаря точно на интерфейса на абстракцията, в действителност двата интерфейса може да бъде различно, което е обикновено за изпълнител.
- **Конкретен изпълнител (ConcreteImplementor):** имплементира интерфейсият изпълнител и определя конкретното му изпълнение.

Примерен код:

```
using System;
namespace DoFactory.GangOfFour.Bridge.Structural { // Bridge Design Pattern.
    class MainApp {
        static void Main() {
            Abstraction ab = new RefinedAbstraction();
            ab.Implementor = new ConcreteImplementorA();
            ab.Operation();
            ab.Implementor = new ConcreteImplementorB();
            ab.Operation();
            Console.ReadKey();
        }
    }

    class Abstraction {
        protected Implementor implementor;
        public Implementor Implementor {
            set {
                implementor = value;
            }
        }
        public virtual void Operation() {
            implementor.Operation();
        }
    }

    abstract class Implementor {
        public abstract void Operation();
    }

    class RefinedAbstraction : Abstraction {
        public override void Operation() {
            implementor.Operation();
        }
    }

    class ConcreteImplementorA : Implementor {
        public override void Operation() {
            Console.WriteLine("ConcreteImplementorA Operation");
        }
    }

    class ConcreteImplementorB : Implementor {
        public override void Operation() {
            Console.WriteLine("ConcreteImplementorB Operation");
        }
    }
}
```


3. Композиция (Composite)

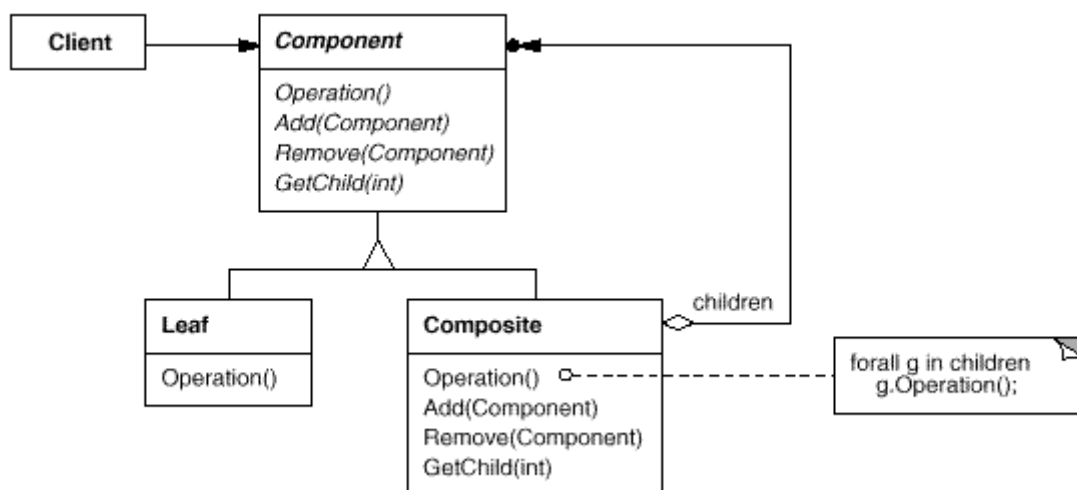
Съставя обектите в дървовидни структури за да представляват част от цяла йерархия. Композицията позволява на клиента да работи с индивидуалните обекти и композиции от обекти по един и същ начин.

Ключът композиционният шаблон е абстрактен клас, който представлява едновременно примитивните и техните контейнери.

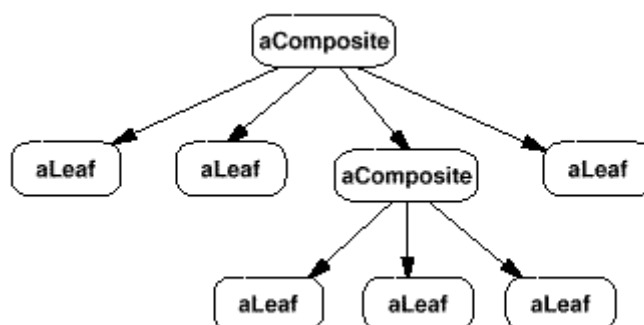
За графичните системи, този клас се нарича график (*Graphic*). Класът график декларира операции като рисуване (*Draw*), които са специфични за графичните обекти.

Използваемост: Композиционният шаблон се използва когато:

- Иискаме да представяме част от цялата йерархия на обекти.
- Иискаме клиенти да могат да игнорират разликата между композиции на обекти и отделни обекти. Клиентите ще третира еднакво всички обекти в композитна структура.



Фиг. 3.1 – Структурата на композиция от структурен шаблон.



Фиг. 3.2 – Структурата на композиция в дървовиден вид.

Участници:

- **Компонент (Component):** Декларира интерфейса за обектите в композицията, прилага поведението по подразбиране за общия интерфейс за всички класове, в зависимост от случая, декларира интерфейс за достъп и управление на своите компоненти.
- **Листо (Leaf):** представлява листа обекти в композицията. Листо няма наследници. Дефинира поведение за примитивни обекти в композицията.
- **Композиция (Composite):** дефинира поведение за компоненти, които имат наследници, съхранява компонентите наследници, прилага операции свързани с наследниците интерфейса Компонент.
- **клиент (Client):** манипулира обекти в композицията чрез интерфейса Компонент.

Примерен код:

```
using System;
using System.Collections.Generic;
namespace DoFactory.GangOfFour.Composite.Structural {
    class MainApp {
        static void Main() {
            Composite root = new Composite("root");
            root.Add(new Leaf("Leaf A"));
            root.Add(new Leaf("Leaf B"));
            Composite comp = new Composite("Composite X");
            comp.Add(new Leaf("Leaf XA"));
            comp.Add(new Leaf("Leaf XB"));
            root.Add(comp);
            root.Add(new Leaf("Leaf C"));
            Leaf leaf = new Leaf("Leaf D");
            root.Add(leaf);
            root.Remove(leaf);
            root.Display(1);
            Console.ReadKey();
        }
    }

    abstract class Component {
        protected string name;
        public Component(string name) {
            this.name = name;
        }
        public abstract void Add(Component c);
        public abstract void Remove(Component c);
        public abstract void Display(int depth);
    }

    class Composite : Component {
        private List<Component> _children = new List<Component>();
        public Composite(string name):base(name) {
        }
        public override void Add(Component component) {
            _children.Add(component);
        }
        public override void Remove(Component component) {
            _children.Remove(component);
        }
        public override void Display(int depth) {
            Console.WriteLine(new String('-', depth) + name);
            foreach (Component component in _children) {
                component.Display(depth + 2);
            }
        }
    }
}
```

```

    }

    class Leaf : Component {
        public Leaf(string name) : base(name) {
        }
        public override void Add(Component c) {
            Console.WriteLine("Cannot add to a leaf");
        }
        public override void Remove(Component c) {
            Console.WriteLine("Cannot remove from a leaf");
        }
        public override void Display(int depth) {
            Console.WriteLine(new String('-', depth) + name);
        }
    }
}

```

4. Декоратор (Decorator)

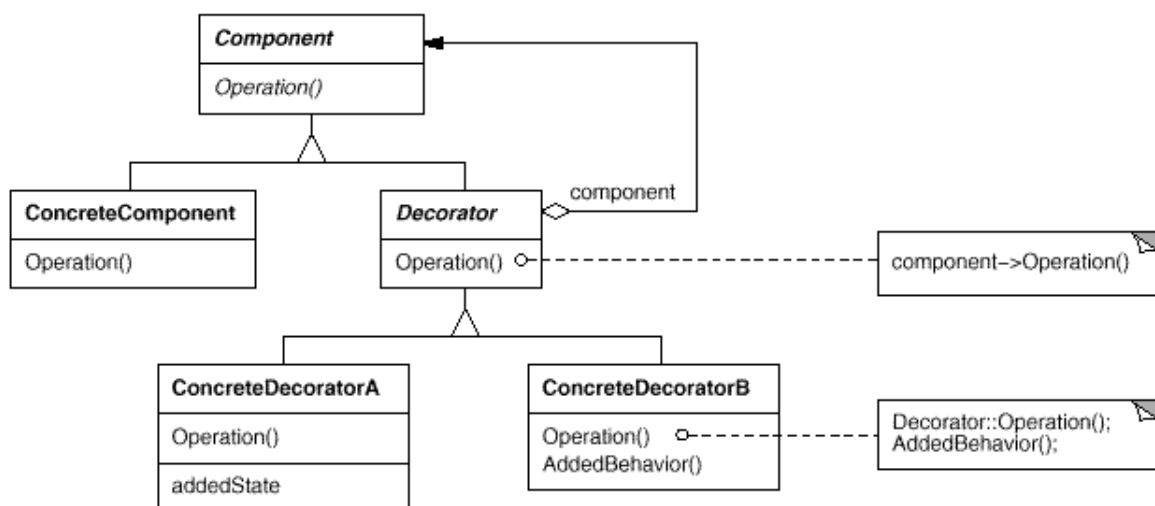
Понякога искаме да добавим отговорности на отделните обекти, а не на целия клас. Едни инструментариум, например, които да позволяват добавянето на свойства като граници или поведение като например преместване към някои частите на потребителския интерфейс.

Един от начините за добавяне на отговорности е с наследяването. Наследяване на граници от друг клас поставя граница зад всяка инстанция на подкласа. Това обаче нее гъвкавост, защото изборът на границата се извършва статично и на определен обект. Клиентът не може да контролира как и кога да украси компонент с границата.

По-гъвкав подход е да се създаде копие на компонента в друг обект, който добавя границата. Обектът създават копието се нарича **декоратор** (*decorator*). Декоратор съвпада с интерфейса на компонента копие, така че неговото присъствие е прозрачно за клиентите на компонента. Декоратора отправя исканията на клиента към този компонент и може да изпълнява допълнителни действия като например изграждане на границата.

Използваемост:

- Добавяне на отговорности на отделните обекти динамично и прозрачно, всичко това без да се засягат други обекти.
- За отговорностите, които могат да бъдат изтеглени.
- Когато разширението чрез подкласиране е непрактично.



Фиг. 4.1 – Структурата на декоратор от структурен шаблон.

Участници:

- **Компонент** (*Component*): дефинира интерфейса за обектите, които могат да имат отговорности добавени към тях динамично.
- **Конкретен компунент** (*ConcreteComponent*): определя обекта, към който допълнителни отговорности могат да бъдат приложени.
- **Декоратор** (*Decorator*): поддържа референция към компунент обект и определя интерфейс, който съответства на интерфейсият компонент.
- **Конкретен декоратор** (*ConcreteDecorator*): добавя отговорности на компонента.

примерен код:

```

using System;
namespace DoFactory.GangOfFour.Decorator.Structural {
    class MainApp {
        static void Main() { // Create ConcreteComponent and two Decorators
            ConcreteComponent c = new ConcreteComponent();
            ConcreteDecoratorA d1 = new ConcreteDecoratorA();
            ConcreteDecoratorB d2 = new ConcreteDecoratorB();
            d1.SetComponent(c); // Link decorators
            d2.SetComponent(d1);
            d2.Operation();
            Console.ReadKey();
        }
    }

    abstract class Component {
        public abstract void Operation();
    }

    class ConcreteComponent : Component {
        public override void Operation() {
            Console.WriteLine("ConcreteComponent.Operation()");
        }
    }
}

```

```

abstract class Decorator : Component {
    protected Component component;
    public void SetComponent(Component component) {
        this.component = component;
    }
    public override void Operation() {
        if (component != null) {
            component.Operation();
        }
    }
}

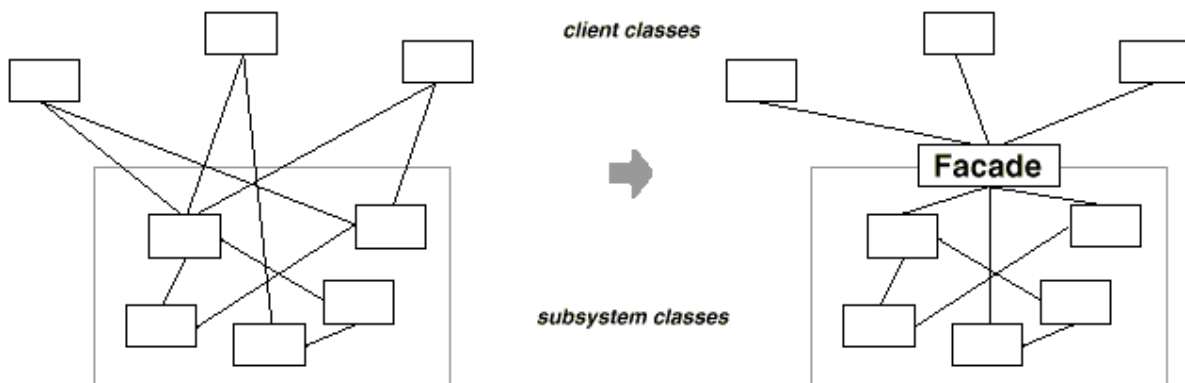
class ConcreteDecoratorA : Decorator {
    public override void Operation() {
        base.Operation();
        Console.WriteLine("ConcreteDecoratorA.Operation()");
    }
}

class ConcreteDecoratorB : Decorator {
    public override void Operation() {
        base.Operation();
        AddedBehavior();
        Console.WriteLine("ConcreteDecoratorB.Operation()");
    }
    void AddedBehavior() {
    }
}
}

```

5. Фасада (Facade)

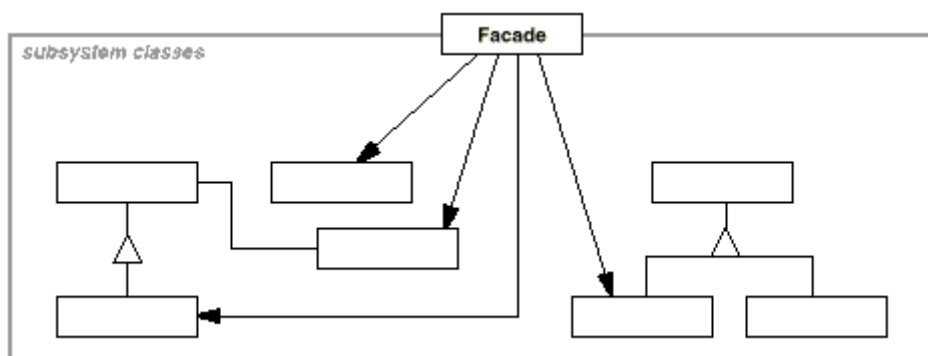
Преструктурирането на системата в подсистеми помага за намаляване на сложността. Общата цел на дизайна е да се сведе до минимум комуникацията и зависимости между подсистемите. Един от начините за постигане на тази цел е да се въведе фасада (*Facade*) обект, който осигурява единен и опростен интерфейс на по-общите улеснения на подсистемата.



Фиг. 5.1 – Фасада наединобект.

Използваемост: Композиционният шаблон се използва когато:

- Искаме да предоставяме прост интерфейс на сложна подсистема.
- Има много зависимости между клиенти и имплементираните класове на една абстракция. Въвежда се една фасада за да се разграничи подсистемата от клиентите и други подсистеми, като по този начин се насърчава на подсистема независимост и преносимост.
- Искаме да поставим един слой на подсистемите. Използва се фасада, за да се определи входна точка за всяко ниво на подсистемата.



Фиг. 5.2 – Структурата на фасада от структурен шаблон.

Участници:

- **Фасада (Facade):** знае кой класове на подсистема са отговорни за искане, делегира клиентски заявки към съответните обекти на подсистемата.
- **подсистемните класове (subsystem classes):** прилагане функционалност на подсистемата, се справят с възложената работа от обекта фасада.

примерен код:

```
using System;
namespace DoFactory.GangOfFour.Facade.Structural {
    class MainApp {
        public static void Main() {
            Facade facade = new Facade();
            facade.MethodA();
            facade.MethodB();
            Console.ReadKey();
        }
    }

    class SubSystemOne {
        public void MethodOne() {
            Console.WriteLine(" SubSystemOne Method");
        }
    }

    class SubSystemTwo {
        public void MethodTwo() {
            Console.WriteLine(" SubSystemTwo Method");
        }
    }
}
```

```

class SubSystemThree {
    public void MethodThree() {
        Console.WriteLine(" SubSystemThree Method");
    }
}

class SubSystemFour {
    public void MethodFour() {
        Console.WriteLine(" SubSystemFour Method");
    }
}

class Facade {
    private SubSystemOne _one;
    private SubSystemTwo _two;
    private SubSystemThree _three;
    private SubSystemFour _four;
    public Facade() {
        _one = new SubSystemOne();
        _two = new SubSystemTwo();
        _three = new SubSystemThree();
        _four = new SubSystemFour();
    }
    public void MethodA() {
        Console.WriteLine("\nMethodA() ---- ");
        _one.MethodOne();
        _two.MethodTwo();
        _four.MethodFour();
    }
    public void MethodB() {
        Console.WriteLine("\nMethodB() ---- ");
        _two.MethodTwo();
        _three.MethodThree();
    }
}
}

```

6. Миниобект (flyweight)

Миниобект е споделен обект, който може да се използва в различни контексти едновременно. Миниобект действа като самостоятелен обект във всеки контекст.

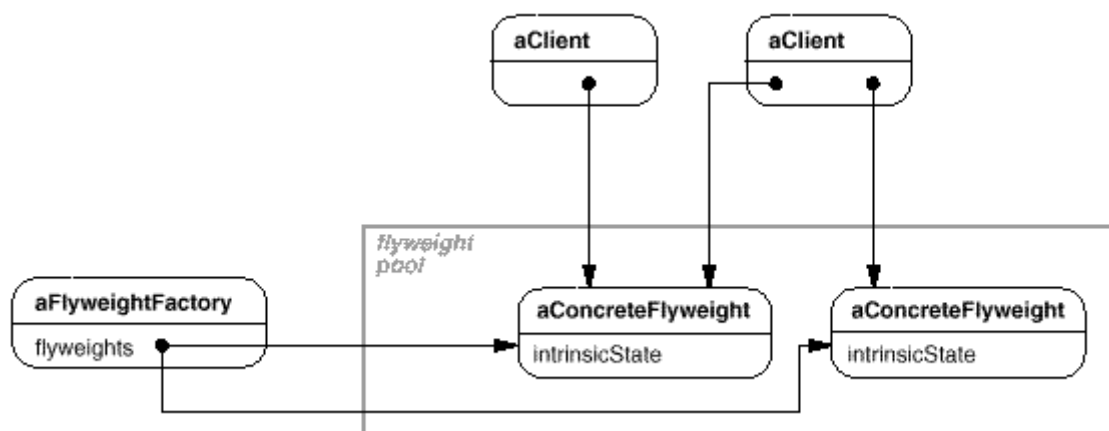
Миниобектите не могат да правят предположения за контекстите, в които те работят. Ключовото понятие тук е разликата между вътрешното и външното състояние. Вътрешното състояние се съхранява в миниобект, тя се състои от информация, която е независима от контекста на миниобект, като по този начин го право споделен. Външното състояние зависи и варира в зависимост от контекста на миниобект и следователно не може да бъде споделено. Клиентските обекти са отговорни за преминаване от вътрешно състояние към миниобект, когато има нужда от него.

Използваемост: шаблонът миниобект се използва когато следните условия са изпълнени:

- Приложението използва голям брой обекти.
- Разходите за съхранение са високи заради огромното количество на обекти.
- Повечето обект за състояние могат да се направят външни.
- Много групи от обекти могат да бъдат заменени със сравнително малко

споделени обекти след премахването на външното състояние.

- Приложението не зависи от идентичността на обекта.



Фиг. 6.1 – Структурата на миниобект от структурен шаблон.

Участници:

- **Миниобект (Flyweight):** декларира интерфейс, чрез които миниобектите могат да получават и действат по вътрешно състояние.
- **Конкретен миниобект (ConcreteFlyweight):** имплементира интерфейса на миниобекта и добавя място за съхранение за вътрешно състояние, ако има такова. Конкретният миниобект трябва да бъде споделен.
- **Несподелен конкретен миниобект (UnsharedConcreteFlyweight):** не всички подкласове на миниобекта трябва да бъдат споделени. Интерфейса на миниобекта позволява споделянето но не го прави задължително. Общата характеристика за обектите на несподелените конкретни миниобекти е, че имат конкретни миниобекти като наследници на някакво ниво в структурата на миниобекта.
- **Фактор на миниобект (FlyweightFactory):** създава и управлява обектите на миниобекта, гарантира, че миниобект са споделени правилно.
- **Клиент (Client):** съдържа референция към миниобекта(ите), изчислява или съхранява вътрешното състояние на миниобекта(ите).

примерен код:

```
using System;
using System.Collections;
namespace DoFactory.GangOfFour.Flyweight.Structural {
    class MainApp {
        static void Main() {
            int extrinsicstate = 22;
            FlyweightFactory factory = new FlyweightFactory();
            Flyweight fx = factory.GetFlyweight("X");
            fx.Operation(--extrinsicstate);
            Flyweight fy = factory.GetFlyweight("Y");
        }
    }
}
```



```

        fy.Operation(--extrinsicstate);
        Flyweight fz = factory.GetFlyweight("Z");
        fz.Operation(--extrinsicstate);
        UnsharedConcreteFlyweight fu = new
        UnsharedConcreteFlyweight();
        fu.Operation(--extrinsicstate);
        Console.ReadKey();
    }
}

class FlyweightFactory {
    private Hashtable flyweights = new Hashtable();
    public FlyweightFactory() {
        flyweights.Add("X", new ConcreteFlyweight());
        flyweights.Add("Y", new ConcreteFlyweight());
        flyweights.Add("Z", new ConcreteFlyweight());
    }
    public Flyweight GetFlyweight(string key) {
        return ((Flyweight)flyweights[key]);
    }
}

abstract class Flyweight {
    public abstract void Operation(int extrinsicstate);
}

class ConcreteFlyweight : Flyweight {
    public override void Operation(int extrinsicstate) {
        Console.WriteLine("ConcreteFlyweight: " + extrinsicstate);
    }
}

class UnsharedConcreteFlyweight : Flyweight {
    public override void Operation(int extrinsicstate) {
        Console.WriteLine("UnsharedConcreteFlyweight: " +
            extrinsicstate);
    }
}
}

```

7. Пълномощно (Proxy):

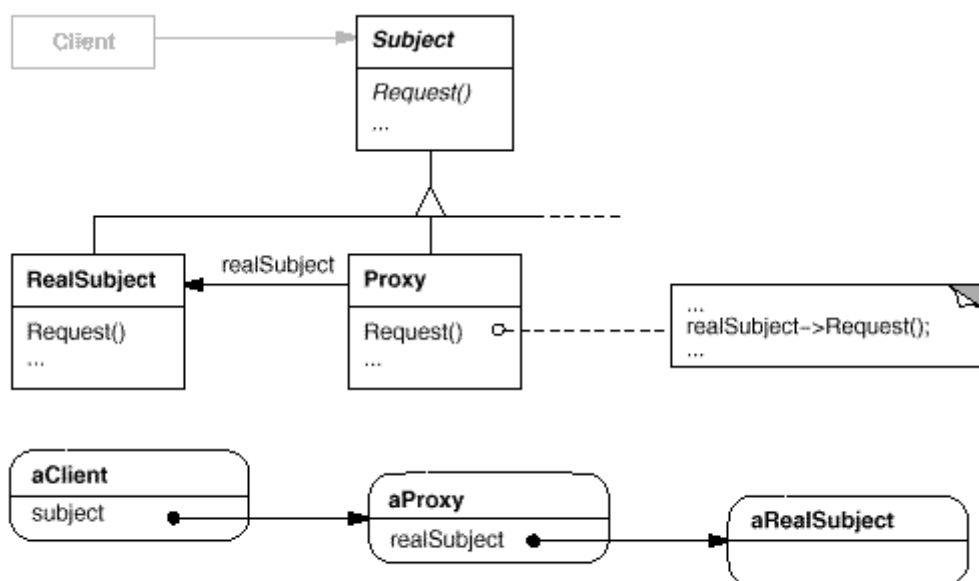
Една от причините за контролиране на достъпа до даден обект е да се отсрочи пълната стойност на неговото създаване и инициализация докато ни се наложи да го използвам. Някои графични обекти, като големите растерни изображения, може да струват скъпо създаването им. Но отварянето на документ трябва да бъде бързо, за това трябва да се избегне създаването на всички скъпи и големи обекти наведнъж, когато се отваря документа. Това така или иначе не е необходимо, тъй като не всички от тези обекти ще бъдат видими в документа в същото време.

Тези ограничения ще предложат създаването на всеки скъп обект **при търсене** (*on demand*), което в този случай ще се случи, когато изображението става видимо.

Решението е да се използва друг обект, **пълномощно изображение** (*image proxy*), който действа като заместител на реалния образ. Пълномощникът действа точно като изображението и се грижи да го инстанцира, когато това е необходимо.

Използваемост: шаблонът пълномощно е приложим при следните ситуации:

- **Дистанционно пълномощно** (*remote proxy*): осигурява локален представител на един даден обект в друго адресно пространство. NEXTSTEP [Add94] използва класа NXProxy за тази цел.
- **Виртуално пълномощно** (*virtual proxy*): създава скъпите обекти при търсенето им.
- **Защитаващо пълномощно** (*protection proxy*): контролира достъпа до оригиналния обект. Защитаващите пълномощни са полезни, когато обектите трябва да имат различни права за достъп.



Фиг. 7.1 – Структурата на пълномощно от структурен шаблон.

Участници:

- **Пълномощно** (*Proxy*): съдържа референция, която позволява на пълномощното достъпа до реалния обект. Предоставя идентичен интерфейс на предмета, така че пълномощника да може да заменят реалния предмет. Контролира достъпа до реалния обект и може да бъде отговорен за създаването и изтриването му.
- **Предмет** (*Subject*): определя общият интерфейс за реалния предмет и пълномощното, така че пълномощното да може да се използва навсякъде, където се намира реалния предмет.
- **Реален предмет** (*RealSubject*): определя реалния и истенския редмет, който представлява пълномощното.

примерен код:

```
using System;
namespace DoFactory.GangOfFour.Proxy.Structural {
    class MainApp {
        static void Main() {
            Proxy proxy = new Proxy();
            proxy.Request();
            Console.ReadKey();
        }
    }

    abstract class Subject {
        public abstract void Request();
    }

    class RealSubject : Subject {
        public override void Request() {
            Console.WriteLine("Called RealSubject.Request()");
        }
    }

    class Proxy : Subject {
        private RealSubject _realSubject;
        public override void Request() {
            if (_realSubject == null) {
                _realSubject = new RealSubject();
            }
            _realSubject.Request();
        }
    }
}
```

Заклучение

Според мен, шаблоните за проектиране са много актуални в обектно-ориантираните езици когато става дума за дизайн защото дава възможност да се отсрани лесно и бързо проблема при разширяване или актуализиране защото структурните шаблони позволяват лесно използване на наследяването в класовете и подкласовете и също така минимизиране на обектите

Шаблоните за дизайн са реализирани на първо време на обектно-ориантирания език C++, в сегашно време те се реализират най-често под езика C# който предоставя много повече възможности за реализация и най-вече заради подобрения потребителски интерфейс за разлика от C++.

Следвайки утвърдената схема за писане на качествени технологични книги с отворен код от големи, но добре организирани авторски екипи (вж. проектите за безплатните книги Intro C#, Intro Java и .NET Framework), Светлин Наков започна работата по нов проект: безплатна книга за шаблони за софтуерен дизайн (design patterns) на български език наречена “Класически шаблони за дизайн”. Проектът има за цел да бъде написана оригинална българска книга за най-често използваните шаблони за софтуерен дизайн (*GoF patterns*), а след това да бъде продължена инициативата като се включат и архитектурни шаблони.

Екипът на проекта се ръководи от Николай Василев, Цветан Василев и Николай Томитов.

Официалният сайт на безплатната книга за шаблони за софтуерен дизайн за момента е в Google Code: <http://code.google.com/p/design-patterns-book/>

Използвана литература:

1. Addison Wesley - Gamma, Helm, Johnson, Vlissides – “Design Patterns, Elements of Reusable Object Oriented Software”, 1998
2. http://sourcemaking.com/design_patterns
3. <http://www.dofactory.com/Patterns/Patterns.aspx/>
4. <http://www.nakov.com/blog/2011/12/27/free-bulgarian-design-patterns-book/>